

NOVA

IMS

**Information
Management
School**

MACHINE LEARNING AND OPERATIONS PROJECT

Master in Data Science and Advanced Analytics

NOVA Information Management School

Universidade Nova de Lisboa

Predictive Modeling of Credit Default

Professor Nuno Rosa



GitHub Repository

https://github.com/vilmanorja/MLOps_project.git

Group Members

Kida de Moraes Aly, 20231491

Rita Serra, 20240515

Vilma Norja, 20242051

2024/2025

Index

1. Introduction	3
2. Data Description and Analytical Objective.....	3
3. Project Planning	3
4. Description of the process and the pipelines	4
4.1. Exploratory Data Analysis (EDA)	4
4.2. Raw Data Expectations	4
4.3. Data Cleaning	4
4.4. Feature Engineering.....	4
4.5. Feature Expectations	5
4.6. Data Split	5
4.7. Preprocessing of Train / Test Data	6
4.8. Feature Selection	6
4.9. Model Selection	6
4.10. Model Training and SHAP Analysis	6
4.11. Model predictions and Results	7
4.12. Data drift and Model Performance Monitoring.....	7
5. Orchestration with Kedro and Prefect	8
6. Model Serving	8
7. Production Implementation and MLOps Considerations.....	8
8. Conclusions.....	9
Appendix A - Tables	10
Appendix B – Figures	11

Table of Figures

Figure 1 List of packages and versions	11
Figure 2 Class Distribution of HasDefault.....	12
Figure 3 Feature selection based on average rank. A lower combined rank means the feature consistently scored higher across the methods, indicating greater relevance.	12
Figure 4 SHAP summary plot of feature importance	13
Figure 5 Age Distribution Over Time with Kolmogorov-Smirnov Drift Alerts	13
Figure 6 Kolmogorov-Smirnov (KS) drift detection for the Avg_Monthly_Income variable over time	14
Figure 7 Kolmogorov-Smirnov (KS) drift detection for the Avg_Monthly_expenses variable over time.	14
Figure 8 Multivariate Drift Detection Using PCA Reconstruction Error	14
Figure 9 Estimated model accuracy over time using CBPE (Confidence-Based Performance Estimation).	15
Figure 10 (a) (b) Structure of the project with Kedro	17

1. Introduction

This project is a proof of concept focused on applying MLOps principles to a real-world problem. We used real customer data to build a system that predicts the *HasDefault* variable, which indicates whether a customer has defaulted on a loan. The project places strong emphasis on utilizing best practices and tools from the MLOps domain to ensure scalability, reliability, and maintainability throughout the machine learning lifecycle.

List of packages and versions are shown in Figure 1.

2. Data Description and Analytical Objective

In this project, we worked with real data from a bank to try to predict which customers might default on their loans. To do this, we had access to several different datasets, each one giving us a part of the full picture:

- The **Transactions dataset** included all transactions made by each customer, with their respective dates.
- The **Funds dataset** gave us monthly snapshots of the amount of funds customers had.
- The **Loans dataset** provided details about the loans we were trying to predict, grouped by creation month (e.g., *loans_20231231.csv* for loans created in December 2023). It also included the target variable *Has_Default*.
- The **Loans history dataset** tracked the monthly status of all loans.
- The **Customers dataset** contained demographic information like age, gender, and location.

We combined all these sources to build the final customer features, which mix behavioral patterns such as average monthly expenses or average monthly funds of the customer and demographic attributes. One of the biggest challenges was avoiding data leakage, ensuring that, for each loan, we only used customer data available up to the loan's creation date. Handling this time alignment correctly was crucial and is explained in more detail later in the report.

The main objective was to predict the *Has_Default* variable, which indicates if a customer defaulted on a loan. Since defaults were relatively rare in our dataset, we faced a class imbalance issue. To evaluate the model properly, we focused on:

- **Recall** – our top priority, since missing a default is riskier than a false alarm.
- **F1-score** – to balance recall and precision.
- **Accuracy** – used as a secondary reference, since it can be misleading in imbalanced cases.

In summary, our approach focused on building a reliable predictive model while respecting the time constraints of real-world data and choosing the right metrics to evaluate performance in a sensitive and imbalanced setting.

3. Project Planning

We organized the project using a scrum-inspired approach, dividing the work into short sprints with clear goals. This helped us manage time well and keep all components integrated. Initially, we assigned tasks at the start of each sprint and met weekly to review progress and plan for the future. In the final two weeks, we switched to check-ins every three days, like Daily Scrums, which helped us decide faster and fix issues quickly. The sprint schedule is summarized in Table 1. We used Git for version control and collaboration, enabling efficient tracking of changes and seamless teamwork throughout the development process.

4. Description of the process and the pipelines

4.1. Exploratory Data Analysis (EDA)

Notebook: *EDA_raw_data* and *EDA_final_features*

We performed the exploratory analysis in two stages to get a clear understanding of the data, both in its raw form and after the feature engineering process. First, we conducted EDA on the raw data to check the quality of the original data, identify important variables, and find problems like missing values. After the feature engineering process, we did another round of EDA to review the distribution of new variables and check for possible outliers or remaining issues. Additionally, we analyzed the distribution of the target variable, which showed a significant class imbalance, as seen in Figure 2. Also, some variables were highly correlated with each other, which can cause multicollinearity. We addressed this issue during the feature selection process, where we kept the most relevant variables for the model and removed the collinear ones. This approach helped us validate the changes made and ensured the data was ready for the next steps in modeling.

4.2. Raw Data Expectations

Pipeline: *ingestion*

The expectations were built in two phases. First, simple checks were applied to the raw data to ensure that data types were as expected and all required columns were present. The validated features were then saved to the data layer *02_intermediate*. The pipeline produces a report where the outcome of each expectation can be reviewed.

4.3. Data Cleaning

Pipeline: *data_cleaning*

During the data cleaning process, we converted all variables with temporal information to the datetime format to make sure that operations involving dates would be consistent and accurate.

- **Transactions** dataset: We filled missing values in the '*CustomerIdCreditNew*' (the customer who received the amount) and '*CustomerIdDebitNew*' (the customer who sent the amount) columns with 0 and converted them to integers. We also removed rows where both fields were 0, ensuring that only valid transactions involving at least one customer were kept.
- **Loans history** dataset: We removed the rows where the '*CreditAmount*' value was missing, to ensure that only complete records remained. For the column indicating the number of installments to be paid, we filled the missing values with 0, assuming that, in the absence of this information, the number of installments would be zero.
- **Loans** dataset: We applied the same cleaning steps as in the Loans history and additionally removed the letter 'M' from values, converting them to integers for consistency.
- **Customer** dataset: We filled in the missing values in the annual net monthly income column with 0 and we removed the rows with missing customer segments to ensure complete and consistent demographic data.

4.4. Feature Engineering

Pipeline: *feature_engineering* and notebooks: *feature_eng_cust_demographics.ipynb* and *feature_eng_customer_behavior.ipynb*

The goal of this phase was to transform the raw data into meaningful features that capture both the behavioral patterns and risk profiles of each customer. These features would form the basis for predicting loan defaults.

To guide this process, we used a configurable parameter called *run_date*, defined in the *parameters.yml* file. This value could be set to a single date or a range of dates, allowing us to process all loans created within a specific period. For example, if *run_date* was set to 20231231, we would process all loans created in December 2023. Likewise, multiple dates could be used to include loans from several months in one run. Importantly, for each loan considered, we ensured that only information available prior to the loan's creation date was used to generate features, such as customer transactions, fund balances, and previous loan history. This temporal filtering was essential to avoid data leakage and simulate a real-world prediction scenario. The main features were built from several sources:

- **Funds History:** We calculated the average monthly balance and standard deviation to assess the customer's financial stability.
- **Transactions:** From this, we estimated the average monthly income and expenses, as well as their variability, to capture financial consistency and spending behavior.
- **Loans dataset:** Contained details about the loan under evaluation, including credit amount, installments, and payment frequency.
- **Loans History:** Helped us derive past loan experience indicators, such as number of previous loans, defaults, and total borrowed amounts—all based on data prior to the loan being predicted.
- **Customer demographics:** Provided context through variables like age, marital status, employment status, risk rating, and net monthly income segment.

All of these features were consolidated into a final dataset—*customer_features.csv*—structured with one row per customer-loan pair. This dataset served as the foundation for model training and evaluation in the subsequent steps.

4.5. Feature Expectations

Pipeline: *features_data_tests*

After the data cleaning and feature engineering pipelines were executed, a second round of expectations was created for the processed dataset. In this phase, more sophisticated expectations were applied to ensure that the values were reasonable and aligned with domain knowledge. For categorical features, we checked that the values belonged to a predefined set, while for numerical features, we verified that the values fell within reasonable ranges. Even more advanced expectations could have been developed, for example, to detect changes in data distributions more effectively. After running the expectations, features were stored in the feature store to improve reusability and guarantee consistency, ensuring that everyone works with the exact same version of the features. The data was initially divided into three groups: numerical features, categorical features, and the target. However, in hindsight, we realized that an alternative grouping, for example by feature type (e.g., demographic features, transaction features, etc.) would have been more meaningful and informative.

4.6. Data Split

Pipeline: *split_data*

The data was split into training and test sets using scikit-learn's *train_test_split* function, with 80% of the data used for training and 20% reserved for testing, as defined in the *parameters.yml* file. Depending on the value of the *load_from_feature_store* parameter, the data is loaded either from the feature store or from the local

data folder. The resulting split datasets are then stored in the *05_model_input* folder.

4.7. Preprocessing of Train / Test Data

Pipeline: *feature_preprocessing_train, feature_preprocessing_test*

After splitting the data into training and test sets, we applied feature scaling to the numeric variables and one-hot encoding to the categorical variables. This preprocessing was performed using a *ColumnTransformer*. The transformer was fitted on the training data and then applied to both the training and test sets to ensure consistency. This approach helps avoid data leakage and ensures that transformations are based solely on information available during training.

4.8. Feature Selection

Pipeline: *feature_selection*

Three different methods were used to select the most relevant features for the model:

- **SelectKBest** was chosen for its simplicity and efficiency, using the F-score to select the features most related to the target variable. The 15 best variables were selected.
- **Lasso** with cross-validation (LassoCV) was used to regularize the variables, penalizing coefficients of less relevant features. This helps reduce the risk of overfitting and multicollinearity, keeping only the most important variables for the model.
- **Recursive Feature Elimination** (RFE) with *RandomForestClassifier* was chosen for its ability to iteratively evaluate the importance of variables, removing the least important ones until the optimal set of features is identified.

After applying the three methods, we selected the features that appeared in at least '*num_methods*' methods. This parameter set to 2 by default was added to provide flexibility in the selection process and defines the minimum number of methods in which a feature must appear to be included in the final selection. After that, we created a combined ranking by averaging the ranks assigned to each feature based on their F-scores, Lasso coefficients, and RFE importances. In this ranking, a lower value indicates higher relevance, meaning the feature consistently performed well across the different selection methods.

To reduce multicollinearity, we checked for highly correlated features (correlation > 0.7) and kept only the one with the best combined rank. This process resulted in the final set of variables (Figure 3) used for training the model, keeping only the most useful and avoiding redundancy.

4.9. Model Selection

Pipeline: *model_selection* and notebook: *model_training*

For model selection, we ran cross-validation using three different classifiers: *RandomForest*, *GradientBoostingClassifier*, and *XGBClassifier*. All experiments were tracked using *MLflow* autologging.

After evaluating performance, *XGBClassifier* emerged as the best-performing model. We then conducted hyperparameter tuning using a grid search (5 folds) based on the parameter combinations defined in *parameter_grid.yml*. If the tuned model achieved a higher *F1-score* on the test set compared to previous experiments, it was saved as the new champion model.

4.10. Model Training and SHAP Analysis

Pipeline: *model_train* notebook: *model_training.ipynb*

Once the best model (*XGBClassifier*) was selected, we trained it using the final training data and registered the trained model.

To explain the model's predictions, we performed a SHAP analysis (Figure 4). The SHAP summary plot showed that the most impactful features were: Credit Type (e.g., Arranged Overdraft), Payment Frequency (e.g., Monthly payments), Average Monthly Funds and Income Stability. These features had the highest influence on the prediction of defaults. For example, customers with an arranged overdraft or monthly payment schedule were more likely to default. Lower average monthly funds and unstable income also contributed to higher default risk. Demographic variables like Age and Employment Status also played a role, though with slightly less impact.

4.11. Model predictions and Results

Pipeline: *model_predict* and notebook: *model_training.ipynb*

The final model was evaluated on the test set. The overall accuracy was 75%, but since loan defaults are less frequent, we focused on recall and *F1-score* to assess performance more meaningfully.

The model identified 81% of the actual defaulters (recall), which is crucial in banking applications where missing a defaulter can be costly. However, the precision for defaulters was only 49%, meaning that over half of the predicted defaulters did not actually default. This relatively low precision indicates a higher rate of false positives, which could lead to unnecessary interventions for low-risk customers.

Despite that, the *F1-score* of 61% reflects a reasonable trade-off between catching defaulters and limiting false alarms. The detailed classification metrics can be found in Table 2.

This performance reflects a risk-aware strategy, better to catch most defaulters (even with some false positives) than to miss them entirely.

4.12. Data drift and Model Performance Monitoring

Pipeline: *data_drift* and notebook: *data_drift.ipynb*

To monitor both data drift and model performance over time, we used *NannyML*. The reference period was defined using the *reference_split_date* parameter set to "2023-09-30", and the data was split accordingly: data before or on this date formed the reference set, and data after it formed the analysis set.

We focused on detecting drift in key columns, categorical variables like *SegGroup* and *AMLRiskRating*, and numerical ones such as *Avg_Monthly_Income*, *Avg_Monthly_expenses*, *CreditAmount*, and *Age*.

For univariate drift detection, we applied statistical tests: *Jensen-Shannon* for categorical variables (threshold 0.1) and *Kolmogorov-Smirnov* for numerical ones (threshold 0.2).

The results showed no significant drift in features like *CreditAmount* and *Age* (Figure 5), whose distributions remained stable over time. However, we observed notable drift in *Avg_Monthly_Income* (Figure 6) and *Avg_Monthly_expenses* (Figure 7), particularly during December, a pattern that makes sense due to seasonal variations like end-of-year bonuses and increased spending.

To assess the combined effect of all features, we applied a multivariate drift test using the *DataReconstructionDriftCalculator*. While the overall structure of data remained consistent, we detected moderate drift between January and March 2024 (see Figure 8). However, this variation appears to be within acceptable bounds and does not raise major concerns.

We also tried to access model performance over time using *NannyML*'s *CBPE* module (Figure 9). During the reference period (until March 2024), model accuracy remained high and stable, averaging above 0.80. However, starting in April 2024, we observed a clear decline in accuracy, dropping below the lower threshold of 0.60 by June. The confidence intervals also widened, indicating greater uncertainty in predictions.

Although no major data drift was found, this drop in accuracy suggests the possibility of concept drift, where the relationship between features and the target variable changes over time, even if the input data distribution stays the same.

5. Orchestration with Kedro and Prefect

For orchestration, we used Kedro to structure our data pipelines in a modular and reusable way. Kedro also provides a visualization tool that helps in understanding the pipeline structure. The structure of the project is shown in Figure 10. Prefect was used to schedule, monitor, and manage pipeline execution.

6. Model Serving

For model serving, we used Docker to containerize our application, ensuring consistent deployment across environments, as well as easy portability and reproducibility. Streamlit was used to create a simple user interface for interacting with and exploring the data, MLflow experiments and runs, and the pipeline structure. It also enabled running pipelines through the UI instead of the command line.

7. Production Implementation and MLOps Considerations

Since this project is currently a proof of concept, several key improvements would be necessary to transition it into a production-ready system. The following outlines how the current setup could be scaled, the benefits of the technologies we used, the risks involved, and possible mitigations.

The project leverages Kedro for structuring data pipelines in a modular and scalable way, while prefect is employed for orchestrating, scheduling, and monitoring pipeline executions, enhancing transparency and operational control. Git provides robust version control and collaboration capabilities, enabling reliable tracking of code changes. Hopsworks acts as the feature store, ensuring feature consistency, improving reusability, and supporting versioning. Great Expectations enforces data validation and quality checks, which builds trust in the outputs of our pipelines. Docker facilitates consistent deployment environments, improving portability and reproducibility. Streamlit serves as an accessible interface, allowing interacting with and exploring model results and experiments without requiring technical expertise. NannyML provides monitoring for data drift and model performance after deployment, which is crucial for maintaining model reliability in production. MLflow is used to manage experiment tracking, model versioning, evaluation, and also serves as a model registry. The modeling is implemented in Python, which is one of the most widely used programming languages in Machine Learning, facilitating better integration with tools, libraries, and team workflows.

However, there are notable risks that need to be addressed for a production environment. The first is scalability, as the current use of Pandas may not efficiently handle larger data volumes expected in production. To mitigate this, migrating data processing pipelines to PySpark would allow distributed computation and improved performance, though this transition is estimated to require additional development time. Another risk lies in the current lack of comprehensive testing. To address this, we propose developing tests for all functions and pipelines to ensure smooth development of production code (running tests before pushing to main). Additionally, the system currently lacks testing and handling for extreme data scenarios, such as cases where the most important features are missing, which can impact model accuracy or break workflows. Monitoring is another area that requires improvement. A robust monitoring framework should be developed to track all pipeline executions, log results, and send alerts in case of failures or anomalies. This ensures early detection of issues and helps maintain operational reliability.

As the system scales, managing changing resource requirements may become challenging. Introducing Kubernetes for container orchestration would provide autoscaling, fault tolerance, and resource distribution, although this would involve additional engineering effort to set up and integrate with the current infrastructure.

Furthermore, implementing automated retraining workflows would be highly beneficial. The system should be able to ingest new data continuously, retrain models in response to data drift, and validate them before deployment. Introducing shadow testing would also allow new model versions to be tested in parallel with the production model without disrupting existing business processes.

To support monitoring and stakeholder communication, the current Streamlit interface can be expanded or complemented with dedicated dashboards. These tools would offer stakeholders a clear view of model predictions, performance metrics, and data drift indicators. Additionally, incorporating explainability tools such as SHAP into the interface would improve transparency and foster trust by helping users understand how the model makes decisions.

8. Conclusions

This proof-of-concept project demonstrates the application of MLOps best practices to a real-world problem: predicting credit defaults using anonymized customer data. By leveraging tools such as Kedro, Prefect, MLflow, Great Expectations, and Hopsworks, we built a modular, traceable, and reproducible machine learning pipeline. Emphasis was placed on engineering and selecting informative features, tracking and comparing models, interpreting predictions, and monitoring data drift with NannyML.

While the current solution establishes a solid foundation, several areas require further development to reach production-readiness. These include expanding test coverage, improving model serving, and implementing automated retraining and deployment workflows. Enhancing monitoring with alerting, and potentially scaling with tools like Kubernetes, will also improve reliability. Overall, this project highlights the value of MLOps in building scalable, maintainable ML systems and outlines a clear path toward a production-ready implementation.

Appendix A - Tables

Table 1 Project timeline by sprint

Sprint	Estimated Duration	Main Objectives
Sprint 1	Week 1	- Definition of the topic and understanding the problem. - Analysis of the project statement and goals. - Initial exploration of data sources.
Sprint 2	Week 2	- Implementation of the expectations (raw data). - Data Cleaning - Logging of issues and inconsistencies - Initialize the report
Sprint 3	Days 1-3 of Week 3	- Feature engineering from funds, transactions, loans, loans history - Split the data and Feature transformation: apply encoding, scaling, etc. - Feature Selection - Model Selection
Sprint 4	Days 4-7 of Week 3	- Model Training + SHAP Analysis - Create Feature Store - Great Expectations for engineered features - Register the final trained model in the MLflow Model Registry - Update the report
Sprint 5	Day 1- 3 of Week 4	- Integrate Pipelines with Prefect - Data Drift
Sprint 6	Day 4-6 of Week 4	- Docker - Finish the report

Table 2 Classification performance metrics per class

Class	Precision (How many predicted were correct)	Recall (How many actuals were found)	F1-score (Balance of both)	Support (Total examples)
No Default (0)	92%	72%	81%	1503
Default (1)	49%	81%	61%	487
Overall			Accuracy: 75%	1990

Appendix B – Figures

```
requirements.txt
1  confluent-kafka>=2.9.0
2  evidently>=0.6.6
3  expectations>=1.0.1
4  great>=0.2.0
5  great-expectations==0.18.12
6  hopsworks==4.2
7  ipykernel>=6.29.5
8  kaleido==0.2.0
9  kedro>=0.19.12
10 kedro-mlflow>=0.14.4
11 kedro-datasets[pandas]>=1.4.0
12 kedro-viz>=11.0.1
13 mlflow>=2.21.0
14 nannyml>=0.13.0
15 openai>=1.70.0
16 optuna>=4.2.1
17 pip>=25.0.1
18 prefect>=2.20.17
19 pytest==8.4.1
20 coverage==7.9.1
21 pytest-cov==6.2.1
22 shap>=0.47.1
23 streamlit>=1.44.1
24 ucimlrepo>=0.0.7
25 vadersentiment>=3.3.2
26 ydata-profiling>=4.16.1
27 streamlit-ydata-profiling>=0.2.1
28 kaleido
```

Figure 1 List of packages and versions

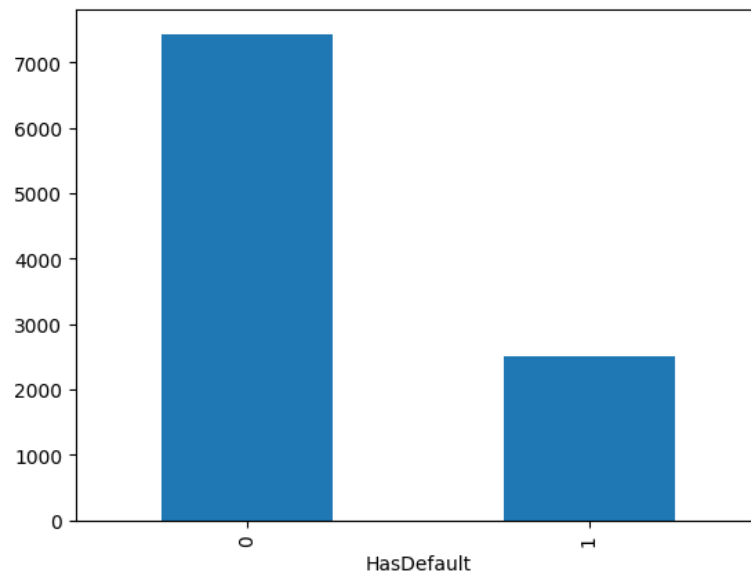


Figure 2 Class Distribution of HasDefault

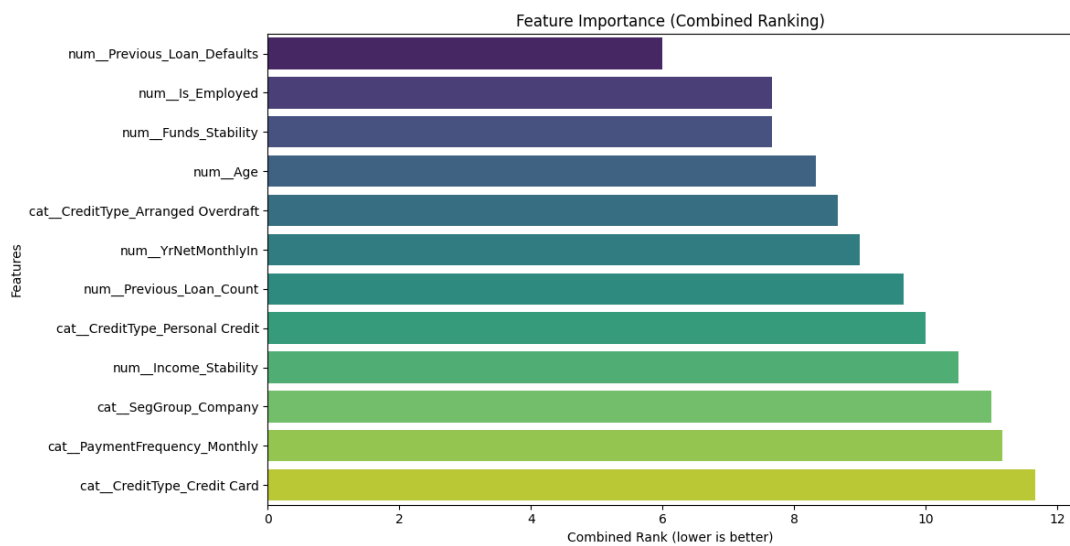


Figure 3 Feature selection based on average rank. A lower combined rank means the feature consistently scored higher across the methods, indicating greater relevance.

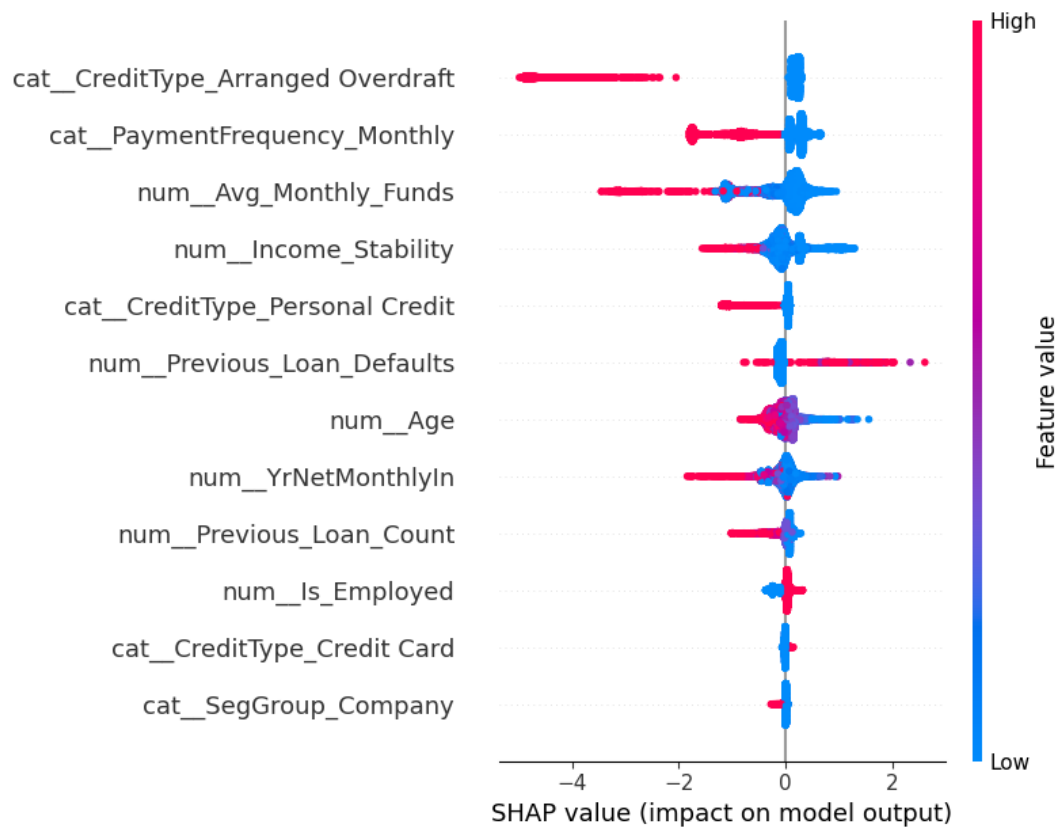


Figure 4 SHAP summary plot of feature importance

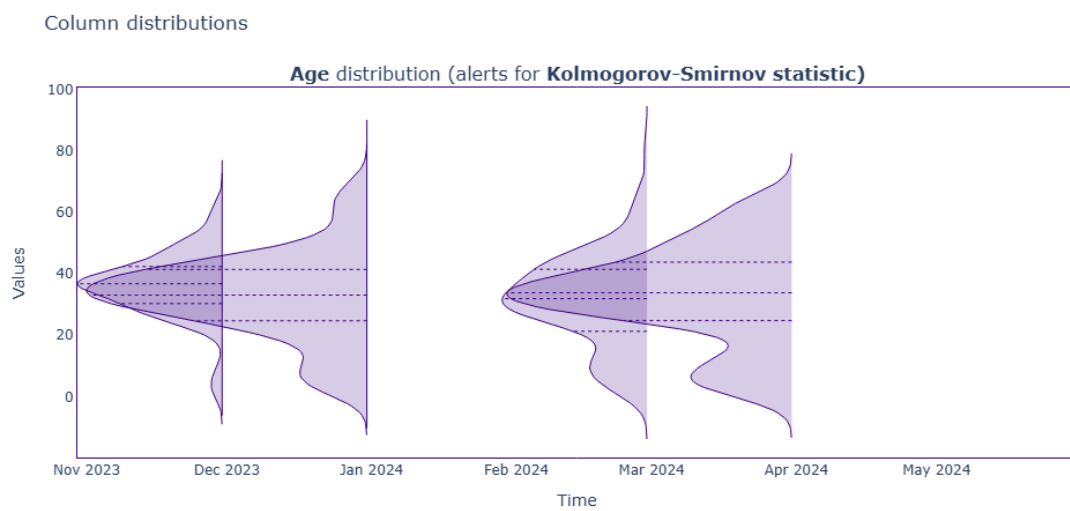


Figure 5 Age Distribution Over Time with Kolmogorov-Smirnov Drift Alerts

Univariate drift metrics

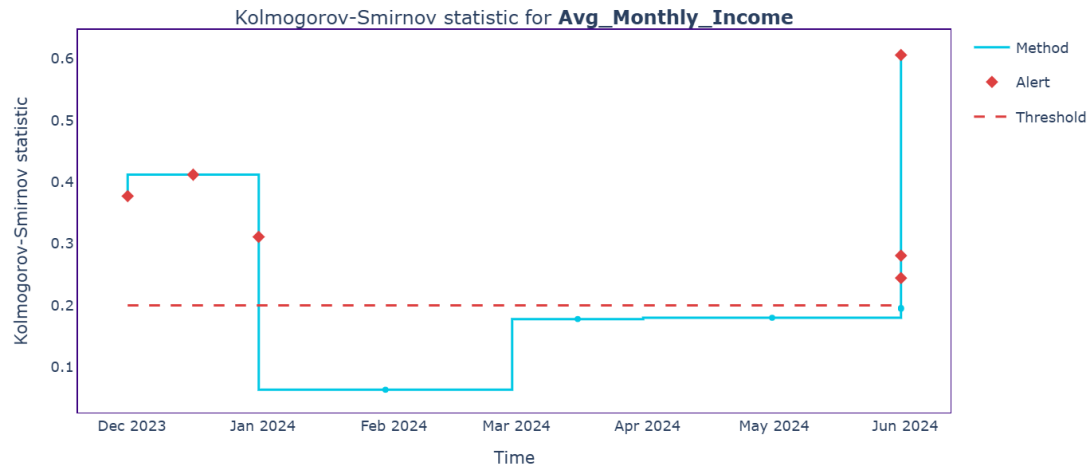


Figure 6 Kolmogorov-Smirnov (KS) drift detection for the Avg_Monthly_Income variable over time

Univariate drift metrics

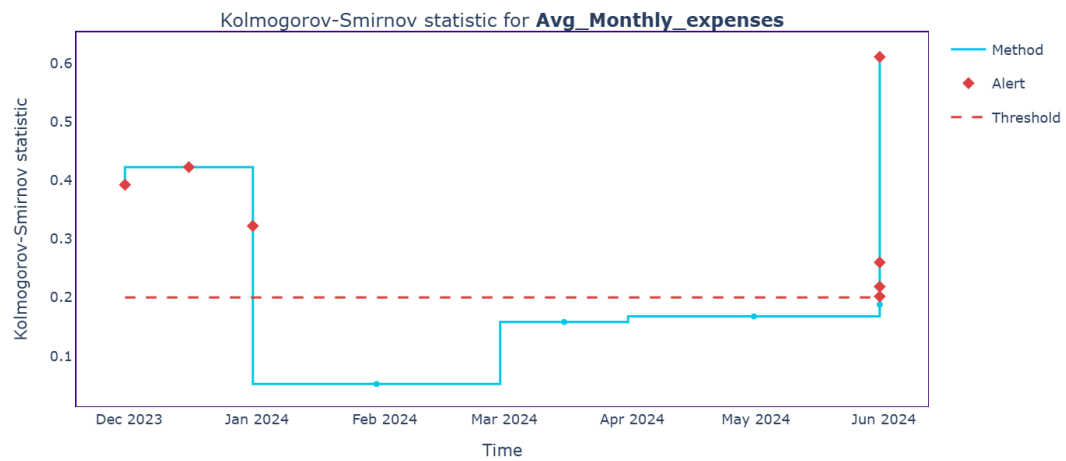


Figure 7 Kolmogorov-Smirnov (KS) drift detection for the Avg_Monthly_expenses variable over time.

Multivariate Drift (PCA Reconstruction Error)

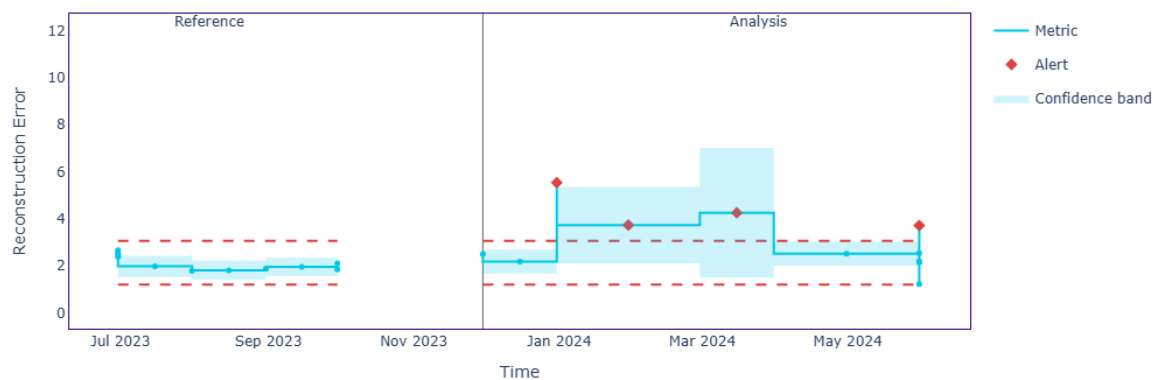


Figure 8 Multivariate Drift Detection Using PCA Reconstruction Error

Estimated performance (CBPE)

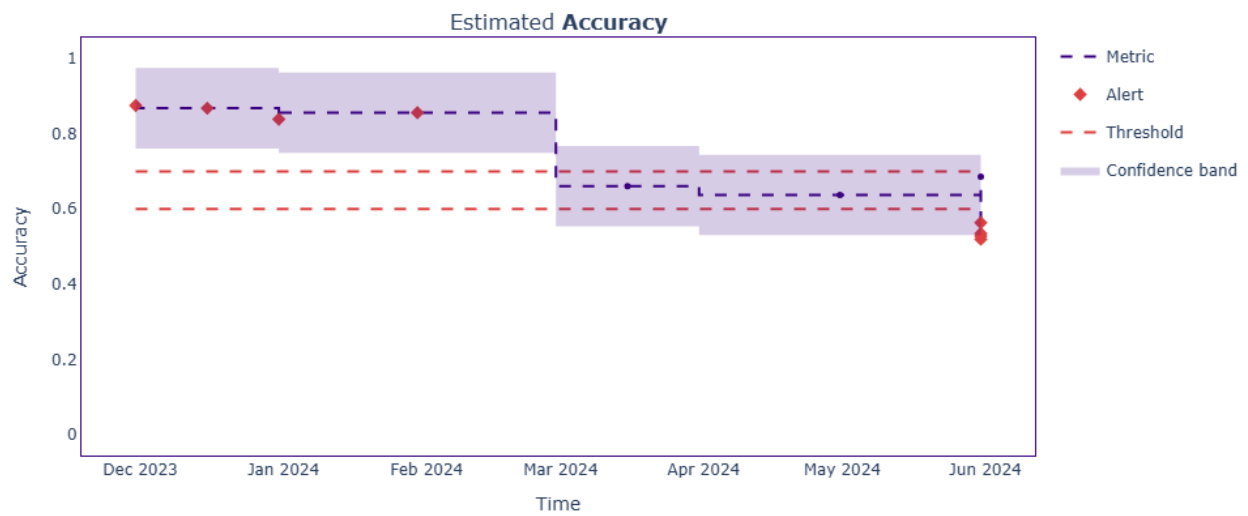
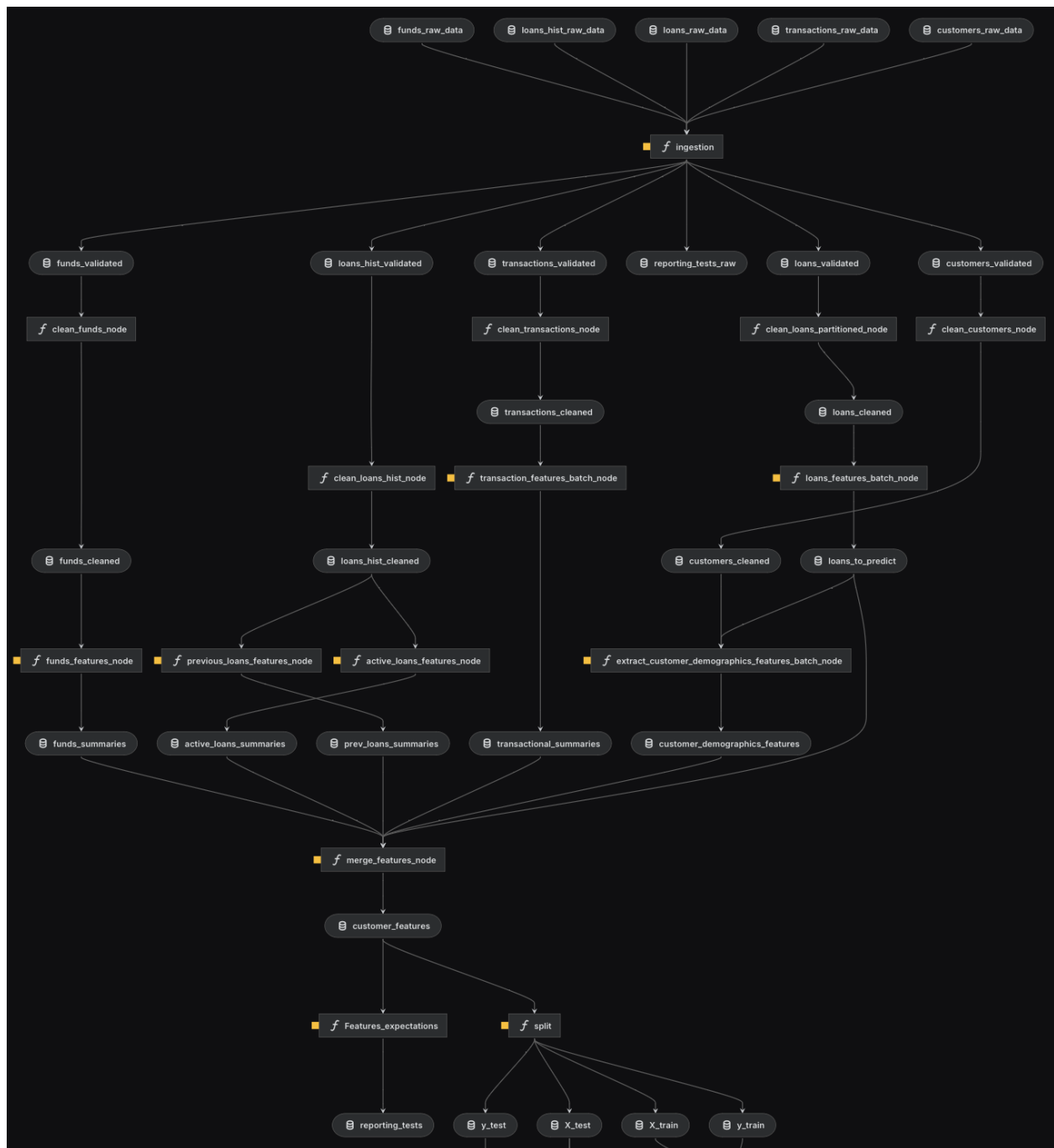
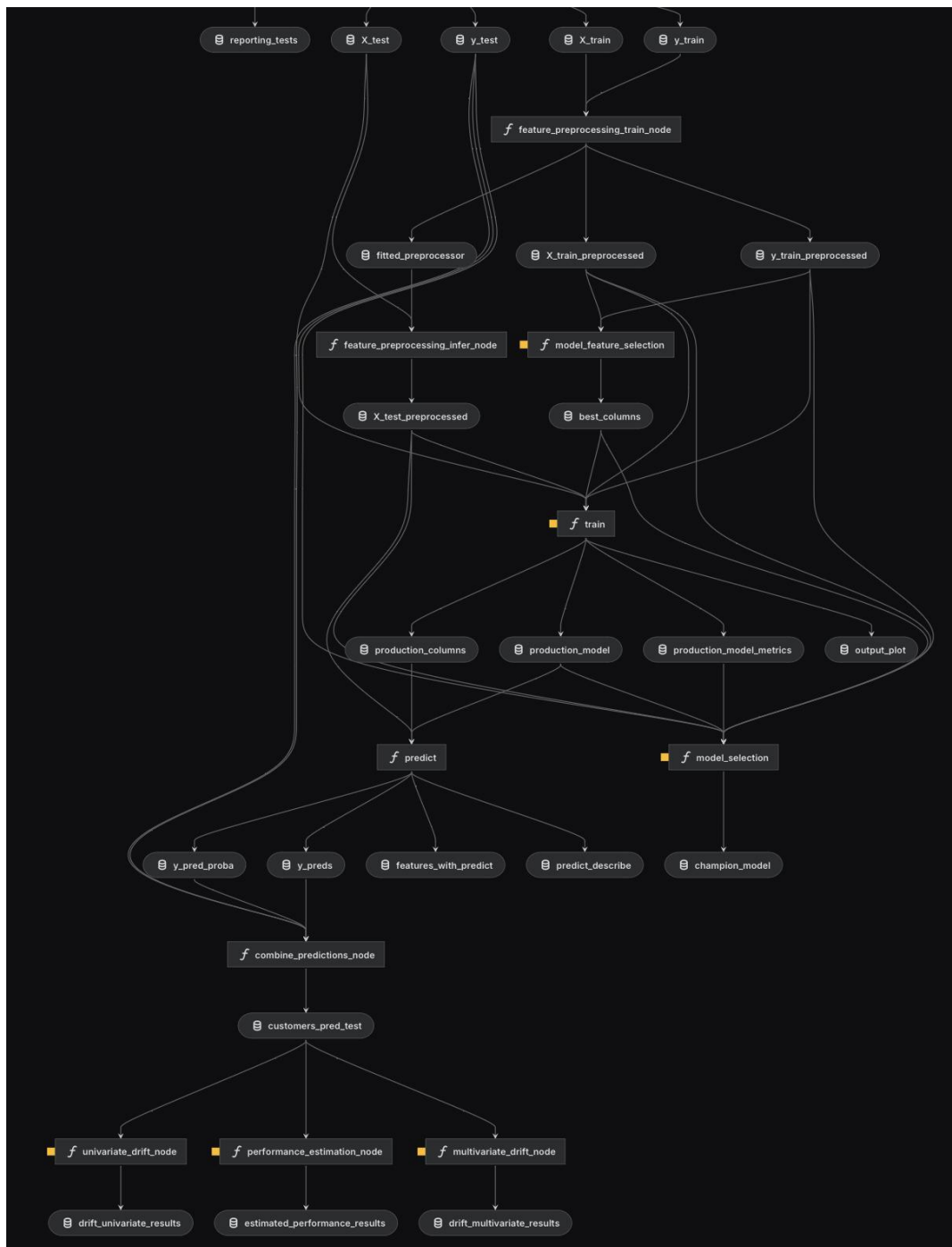


Figure 9 Estimated model accuracy over time using CBPE (Confidence-Based Performance Estimation).



(a) First Part of the structure



(b) Second Part of the structure

Figure 10 (a) (b) Structure of the project with Kedro